

Week 2 - Day 6a: Reviewing Complex Data Types and Functions

We will start easy this week by reviewing some fundamental concepts from week 1 - these include:

- Dictionaries
- Functions
- Indentations
- Variable Scope
- Print vs Return

Exercise 1: Accessing and Reassigning Dictionaries []

Given the following dictionary `numbers`

```
numbers = {  
    "asir": "ten",  
    "hulet" : "two",  
    "amist" : "five",  
    "haya" : "twenty",  
    "arba" : "thirty"  
}
```

1.1 Print the translation of `asir`

```
# TODO: Your code here
```

1.2 Change the value belonging to the key `arba` to its English translation

```
# TODO: Your code here
```

1.3 Add three more numbers to the dictionary

```
# TODO: Your code here
```

1.4 Delete the entries for `"hulet"` and `"haya"`

```
# TODO: Your code here
```

1.5 Check if the key `"hulet"` exists in the dictionary.

- If it exists, print it's value.
- Else, **print:** `"No, it doesn't exist."`

```
# TODO: Your code here
```

1.6 Prints all keys and their values

```
# TODO: Your code here
```

Exercise 2: Creating Dictionaries []

2.1 Write code that creates a dictionary whose **keys** are the **numbers from 1 to 25** and whose **values** are the **squares** of these numbers.

```
# TODO: Your code here
```

Exercise 3 : List to Dictionaries []

In the following code block, each entry in the list is a list containing the name of a person as the first element, and their favorite food as the second.

```
favourite_foods = [  
    ["Nil Gustafsson", "Oven pancakes"],  
    ["Amanuel Temesgen", "Tibs"],  
    ["Mihret Agegnehu", "Tibs"],  
    ["Lwam Zemikael", "Chicago-style Pizza"],  
    ["Alexander Cai", "Tomato Egg Noodles"],  
    ["Matthew", "Nasi Lemak"],  
    ["Bontu Fufa", "Shiro"],  
    ["Eyob Asfaw", "Tibs"],  
    ["Tamiru Alemnew", "Kitfo"],  
    ["Biniyam Aschalew", "Koshari"],  
    ["Dawit", "Mandi"],  
    ["Biruk Mekonen", "Injera with Doro Wat"],  
    ["Joseph Birara", "Gored Gored"],  
    ["Medhanie Irgau", "Doro wet"],  
    ["Betelhem Haile", "Tibs"],  
    ["Biniyam Lombe", "Dulet"],  
    ["Senem Isik", "Chicago-style Pizza"],  
    ["Nick Jiang", "Bimbimbap"],  
    ["Naol Kasinet", "Plov"],  
    ["Meron Kiflay", "Caesar Salad"],  
    ["Makda", "Fondue"],  
    ["Carles Mele", "Croquetas"],  
    ["Natneam Mesele", "Injera with Doro Wat"],  
    ["Param Nagda", "Tibs"],  
    ["Sonika Vuyyuru", "Butter Chicken"],  
    ["Daniel Nieto", "Cuban Rice"],  
    ["Mihret Tekalgn", "Injera with Doro Wat"],  
    ["Nardos Wehabe", "Tibs"],  
]
```

```

    ["Hana Workneh", "Kitfo"],
    ["Abdishakur Yusuf", "Butter Chicken"],
    ["Saba Zerefa", "Shiro"],
]

```

3.1 Convert the list of lists `favourite_foods` to a dictionary, called `favourite_foods_dic` mapping names to foods.

```
# TODO: Your code here
```

3.2 Use the dictionary to print Bontu's favourite food.

```
# TODO: Your code here
```

Exercise 4: String to Dictionaries []

Implement the function that takes a word, and creates a dictionary whose **keys** are each letters of the word and the **values** are the **last position where the letter appears** in the word. For example, for the word `dorowot` the resulting dictionary is `{ "d" : 0, "o" : 5, "r" : 2, "w" : 4, "t" : 6}`. Use zero-indexing.

```

# TODO: Your code here
def letter_last_pos(word):
    pass

# Use this to test your function
assert letter_last_pos("dorowot") == {'d': 0, 'o': 5, 'r': 2, 'w': 4, 't': 6}
assert letter_last_pos("hello") == {'h': 0, 'e': 1, 'l': 3, 'o': 4}
assert letter_last_pos("") == {}

print("All tests pass. []")

```

```

-----
-----
AssertionError                                Traceback (most recent call
last)
Cell In[8], line 2
      1 # Use this to test your function
----> 2 assert letter_last_pos("dorowot") == {'d': 0, 'o': 5, 'r': 2,
'w': 4, 't': 6}
      3 assert letter_last_pos("hello") == {'h': 0, 'e': 1, 'l': 3,
'o': 4}
      4 assert letter_last_pos("") == {}

AssertionError:

```

Exercise 5: Indentation []

So, last week, understanding indentation in Python was very helpful for dealing with **loops** and **conditionals**. The indentation of the lines of code can determine whether it is part of a function body, a loop body, or a conditional body. This is important because it can change the behavior of the code.

Now, given the list of lists of favourite foods of TA's, consider these two versions of the function that finds the favorite dish of a TA:

```
# Version 1
def find_favorite_dish(ta):
    for preference in favourite_foods:
        if preference[0] == ta:
            return preference[1]
    return "The favorite dish of " + ta + " was not found :("

# Version 2
def find_favorite_dish(ta):
    for preference in favorite_foods:
        if preference[0] == ta:
            return preference[1]
    return "The favorite dish of " + ta + " was not found :("
```

Notice, how in the second implementation the `return` statement is indented differently.

6.1 What would happen if he tests both versions by calling `find_favorite_dish("Biniyam Lombe")`?

6.2 Test your guess by calling `find_favorite_dish("Biniyam Lombe")` for both versions.

```
# TODO: Your code here
```

Exercise 6: Scope []

Each function has a (local) scope that is specific to it. Everything outside of functions is referred to as global scope. Variables declared inside a function exist only inside that function's scope and cannot be used outside of it. Let's take a look.

Saba also wants to make a program that can greet other persons. Once it gets the name of the person it should say hi and ask for their age. She tries to write the following code but it keeps failing 😞.

```
def greet(name):
    print("Hi " + name)

def ask_age(name):
    age = int(input(name + " , how old are you?: "))
```

```
greet("Hellina")
ask_age("Hellina")
print("Okay, now I know you're " + str(age) + " years old")
```

7.1 What do you think would happen when the last three lines of this code are run? Explain why.

7.2 Fix the code so that it works as intended: Make sure that `ask_age` returns a value, and that you save it in the right variable when calling `ask_age("Hellina")`

```
# TODO: Your code here
```

Exercise 7: Print vs. Return []

Remember the function that computes the area of a circle from last week? It was implemented as follows:

```
def area_circle(radius):
    return 3.14 * radius ** 2
```

We can **call** this function by giving it a number (or a variable, or an expression) as a radius.

```
print(area_circle(10))
x = 1
print(area_circle(x))

def area_circle(radius):
    return 3.14 * radius ** 2

def print_area_circle(radius):
    print(3.14 * radius ** 2)
```

8.1 What do you think would happen when you run the code below?

```
area = area_circle(10)
print(area)

# TODO: Your code here
```

8.2 What do you think would happen when you run the code below?

```
area = print_area_circle(10)
print(area)

# TODO: Your code here
```

Remember the donut.

Its area is the **area of the outer circle minus the area of the inner circle**.

For example, if `r_outer = 10` and `r_inner = 5`, we can compute its area as follows:

```
area_outer = area_circle(10)
area_inner = area_circle(5)
print(area_outer - area_inner)
```

8.3 Try to do the same computation using the `print_area_circle` function. **Write your code below.**

```
# TODO: Your code here
```

8.4 What happened when you run the code? Why?

Exercise 8: Functions Calling Each Other []

We can define functions that make use of other functions. The volume of a cylinder of radius r and height h is

$$\pi r^2 h$$

In other words, it is the **area of the circle, multiplied with h** . Because we already know how to compute the area of a circle, we use it here:

```
def volume_cylinder(radius, height):
    return area_circle(radius) * height
```

We **call** our function to compute the area of a cylinder with radius 10 and height 1.

```
print(volume_cylinder(10, 1))
```

9.1 Call your function for 3 different arguments to compute the area of 5 different cylinders!

```
# TODO: Your code here
```

9.2 Write a function `volume_cone` that computes the volume of a cone. The volume of a cone with radius r and height h is $h/3$ **times the area of the circle** at the base.

```
# TODO: Your code here
```

9.3 Call your function for 3 different arguments to compute the area of 5 different cones!

Example: `print(volume_cone(10, 1))` --> This should give 104.66666666666667

```
# TODO: Your code here
```

9.4 Fill in the function to compute the **area of the donut** below. Do this by calling the `area_circle` function (twice!).

```
# TODO: Your code here
def area_donut(r_outer, r_inner):
    pass
```

9.5 Call your function for 3 different arguments to compute the area of 5 different donuts!

Example: `print(area_donut(10, 5))` --> This should give 235.5

```
# TODO: Your code here
```

9.6 Write a function `volume_hollow_cylinder` to compute the volume of a hollow cylinder: it is like a 3-dimensional donut, and its volume is the **area of the donut at the base times its height**.

$v = v_{\text{outer}} - v_{\text{inner}}$

The arguments of your function should be `r_outer`, `r_inner` and `height`, and it should make use of the function `area_donut`.

```
# TODO: Your code here
```

9.7 Call your function for 3 different arguments to compute the area of 5 different hollow cylinders!

Example: `print(volume_hollow_cylinder(10, 5, 10))` --> This should give 2355.0

```
# TODO: Your code here
```

Exercise 9: Armstrong Numbers []

Your CS professor, Huy, loves Armstrong numbers and asks you to help identify some Armstrong numbers for them. A three digit integer is said to be an **Armstrong** number if the sum of the cubes of its digits is equal to the number itself. For example, 371 is an Armstrong number since $3^3 + 7^3 + 1^3 = 371$.

10.1 Define a helper function named `extract_digits` that is going to take in a positive integer and then returns the individual digits within the integer as a list. For example, if you give it **371** it should return the list `[1, 3, 7]`. Your list must be sorted in ascending order. Use the built-in function `sorted` to sort the list.

```
# TODO: Your code here
def extract_digits(n):
    pass

# Use this code to test your function
assert extract_digits(371) == [1, 3, 7]
```

```
assert extract_digits(985) == [5, 8, 9]
assert extract_digits(652) == [2, 5, 6]

print("All tests pass. ☑")
```

10.2 Define a helper function named `sum_digit_cubes` that is going to take in a list and it should cube (raise the number by the power 3) and then sum them up. For example, if you're given the list `[3, 7, 1]` it should return the result of $3^3 + 7^3 + 1^3$ which in this case is 371.

```
# TODO: Your code here
def sum_digit_cubes(nums):
    pass

# Use this code to test your function
assert sum_digit_cubes([3, 7, 1]) == 371
assert sum_digit_cubes([4, 2, 3]) == 99

print("All tests pass. ☑")
```

10.3 Define a helper function named `is_armstrong` that is going to take in a number and returns True if the number is an Armstrong number otherwise return False. You should use the `extract_digits` and `sum_digits_cube` functions you defined earlier.

```
# TODO: Your code here
def is_armstrong(num):
    pass

# Use this code to test your function
assert is_armstrong(371) == True
assert is_armstrong(372) == False
assert is_armstrong(153) == True
assert is_armstrong(407) == True
assert is_armstrong(370) == True

print("All tests pass. ☑")
```

10.4 Define a function named `find_armstrong_nums` that finds all 3 digit armstrong numbers. It should return a list of three digit armstrong numbers.

```
# TODO: Your code here
def find_armstrong_nums():
    pass

# Use this code to test your function
assert len(find_armstrong_nums()) == 4

print("All tests pass. ☑")
```

Exercise 10: Perfect Numbers []

A perfect number is a number whose proper divisors add up to the number itself. For example, 28 is a perfect number, because its proper divisors are 1, 2, 4, 7, 14 and $1+2+4+7+14=28$.

Write a function that returns a list containing all the perfect numbers less than 10000.

11.1 Write a helper function `find_divisors` that returns the list of divisors of a number

```
# TODO: Your code here
def find_divisors(n):
    pass
```

11.2 write a function `is_perfect` that checks if a number is perfect number or not using the `find_divisors` function

```
# TODO: Your code here
def is_perfect(n):
    pass

# Use this code to test your function
assert is_perfect(6) == True
assert is_perfect(28) == True
assert is_perfect(29) == False

print("All tests pass. []")
```

Exercise 11: Most Frequent Letter []

Write a function `most_frequent` that takes a `word`, and returns its most frequent letter. For example:

```
most_frequent("addis")      == "d"
most_frequent("addisababa") == "a"
```

Hint: First write a function that takes a word and computes and returns a dictionary whose keys are the letters of the word and its values are how often the letter appears.

```
# TODO: Your code here

# Use this code to test your function
assert most_frequent("addis") == "d"
assert most_frequent("addisababa") == "a"
assert most_frequent("dorowot") == "o"

print("All tests pass. []")
```

Exercise 12: Amharic Word Addition []

Write a function `amharic_add` that takes **two Amharic number words** and returns the **sum of their numeric values**.

Supported Amharic Words:

- `"hulet"` = 2
- `"amist"` = 5
- `"asir"` = 10
- `"haya"` = 20
- `"arba"` = 30

If either word is not recognized, return: `"Invalid input"`.

Hint : Use a dictionary to look up the numeric values of the words.

```
# TODO: Your code here
def amharic_add(word1, word2):
    pass

assert amharic_add("hulet", "asir") == 12
assert amharic_add("amist", "haya") == 25
assert amharic_add("asir", "unknown") == "Invalid input"

print("All tests pass. []")
```

Cell In[2], line 3

Write a function ``amharic_add`` that takes ****two Amharic number words**** and returns the ****sum of their numeric values****.

^
SyntaxError: invalid syntax